CAHIER DES CHARGES

PLATEFORME WEB DE TÉLÉ-ENSEIGNEMENT ET DE SUIVI PÉDAGOGIQUE

1. PRÉSENTATION DE VOTRE PROJET

Contexte : Le projet s'inscrit dans le cadre d'un stage. Il répond à un besoin identifié dans le secteur de l'enseignement supérieur où le suivi des étudiants repose encore largement sur des méthodes traditionnelles présentant plusieurs limites :

- Les supports pédagogiques sont transmis par email, clés USB ou réseaux sociaux sans traçabilité ni organisation
- L'identification des étudiants en difficulté est tardive et intuitive : l'enseignant ne dispose d'aucun indicateur objectif avant les résultats des examens finaux
- La gestion des devoirs (distribution, collecte, vérification des délais) est manuelle et chronophage
- Les ressources sont dispersées sur plusieurs canaux, compliquant l'accès pour les étudiants
- L'étudiant n'a aucune visibilité sur sa progression par rapport au reste de la classe
- Il n'existe pas de vue centralisée sur les performances globales, le taux de participation ou les indicateurs de réussite par filière

Rôle du projet web : Ce projet constitue le cœur de la stratégie numérique envisagée : proposer une plateforme centralisée exploitant l'Intelligence Artificielle pour digitaliser le processus éducatif, automatiser le suivi pédagogique et anticiper les risques d'échec scolaire.

Ressources humaines allouées : Un développeur stagiaire (conception, développement frontend et backend, intégration IA), encadré par un tuteur de stage.

1.1. Les objectifs du site

Un logiciel en ligne — Application web de type LMS (Learning Management System) intégrant un assistant pédagogique prédictif basé sur l'Intelligence Artificielle.

Objectifs principaux :

1. Centraliser l'ensemble des ressources pédagogiques (PDF, vidéos MP4) dans un espace unique

2. **Gérer les devoirs avec rigueur :** création avec dates limites, soumission par l'étudiant, blocage automatique après échéance, notation par l'enseignant
3. Prévenir l'échec scolaire grâce à un assistant IA analysant les données de performance pour identifier les étudiants à risque et proposer des actions correctives
4. Fournir des tableaux de bord avec statistiques visuelles adaptées à chaque rôle (admin, enseignant, étudiant)
5. Sécuriser les accès via une authentification JWT et un contrôle d'accès strict basé sur les rôles

1.2. La cible adressée par le site

La plateforme cible le milieu de l'enseignement supérieur. Trois segments d'utilisateurs sont identifiés :

Priorité	Segment	Caractéristiques	Centres d'intérêt
1	Étudiants	Apprenants de l'enseignement supérieur	Accéder aux cours, télécharger les supports, visualiser les PDF en ligne, soumettre les devoirs, consulter ses notes et sa progression
2	Enseignants	Professeurs, formateurs	Déposer des cours et supports, créer des devoirs avec dates limites, noter les soumissions, identifier les étudiants à risque via l'IA
3	Administrateurs	Personnel technique/administratif	Gérer les comptes, configurer les classes, consulter les statistiques globales et les indicateurs de réussite

1.3. Objectifs quantitatifs

- **Volume de trafic visé :** La plateforme doit supporter plusieurs centaines d'utilisateurs simultanés, particulièrement lors des périodes de soumission de devoirs.
- **Volume d'activité :** Gestion simultanée de multiples cours, supports pédagogiques et devoirs par classe.
- **Performance attendue :**

- **Temps de chargement des pages :** < 3 secondes
- **Temps d'analyse IA (prédiction de risque) :** < 15 secondes avec indicateur de chargement visuel (spinner)

1.4. Périmètre du projet

- **Multilingue :** Non. Le site sera développé en français uniquement dans cette première version.
- **Adaptation mobile :** Oui, le site devra être responsive et adapté aux supports mobiles (tablettes, smartphones).

1.5. Description de l'existant

- **Précédente version du site :** Non, il s'agit d'une création totale.
- **Nom de domaine :** Non encore acquis ; à définir ultérieurement.
- **Hébergement :** À définir. Nécessite un serveur compatible Java/Spring Boot avec base de données PostgreSQL.
- **Documents de présentation :** Les spécifications sont décrites dans le présent cahier des charges.
- **Ressources disponibles :** Aucun logo, élément graphique ou texte préexistant. Les contenus pédagogiques seront fournis par les enseignants.

2. DESCRIPTION GRAPHIQUE ET ERGONOMIQUE

2.1. Charte graphique

- **Code couleur :** À définir lors de la phase de conception. L'univers graphique devra être professionnel, moderne et adapté au contexte éducatif. Couleurs dominantes envisagées : bleu (confiance, sérieux) avec des accents verts (éducation, croissance).
- **Variations de couleur :** Oui, les trois espaces utilisateurs (Administrateur, Enseignant, Étudiant) seront différenciés visuellement par des codes couleur distincts dans les tableaux de bord. Code couleur spécifique pour la prédiction de risque d'échec (vert = Faible → orange = Modéré → rouge = Élevé).
- **Déclinaisons du logo :** Un logo sera créé pour la plateforme. Pas de déclinaison prévue par section.

- **Éléments visuels à réutiliser :** Aucun.
- **Police souhaitée :** Police moderne, lisible et web-friendly (ex. : Roboto, Open Sans, Inter). Doit assurer une bonne lisibilité sur écran, y compris sur mobile.

2.2. Design

- **Exigences de design :** Design moderne et épuré, de type Material Design / Flat Design, cohérent avec l'écosystème Angular Material.
- Interface intuitive et ergonomique, avec une navigation claire adaptée à chaque profil.
- Tableaux de bord visuels avec graphiques et indicateurs de performance.
- Code couleur visuel pour la prédiction de risque d'échec (vert = Faible → orange = Modéré → rouge = Élevé).
- Loader (spinner de chargement) affiché lors des opérations longues (upload de fichiers lourds, analyse IA).
- **Sites de référence :**
    - Moodle (pour la structure LMS)
    - Google Classroom (pour la simplicité d'interface)
    - Canvas LMS (pour l'ergonomie générale)

2.3. Maquettes

Des maquettes graphiques seront réalisées lors de la Phase 1 du projet (Cadrage & Conception) pour les pages principales suivantes :

- Page de connexion / authentification
- Tableau de bord Administrateur (statistiques globales, activité récente)
- Tableau de bord Enseignant (avec zone d'alerte étudiants à risque, taux de soumission)
- Tableau de bord Étudiant (avec vue d'ensemble : cours disponibles, devoirs à rendre, historique des notes, moyenne générale)
- Page de création / gestion de cours
- Page de dépôt de supports (PDF, vidéos MP4)

- Page de consultation de cours avec visualiseur PDF en ligne
- Page de gestion des devoirs (enseignant)
- Page de soumission de devoir (étudiant)
- Page de notation des soumissions (enseignant)
- Interface de l'assistant pédagogique prédictif IA

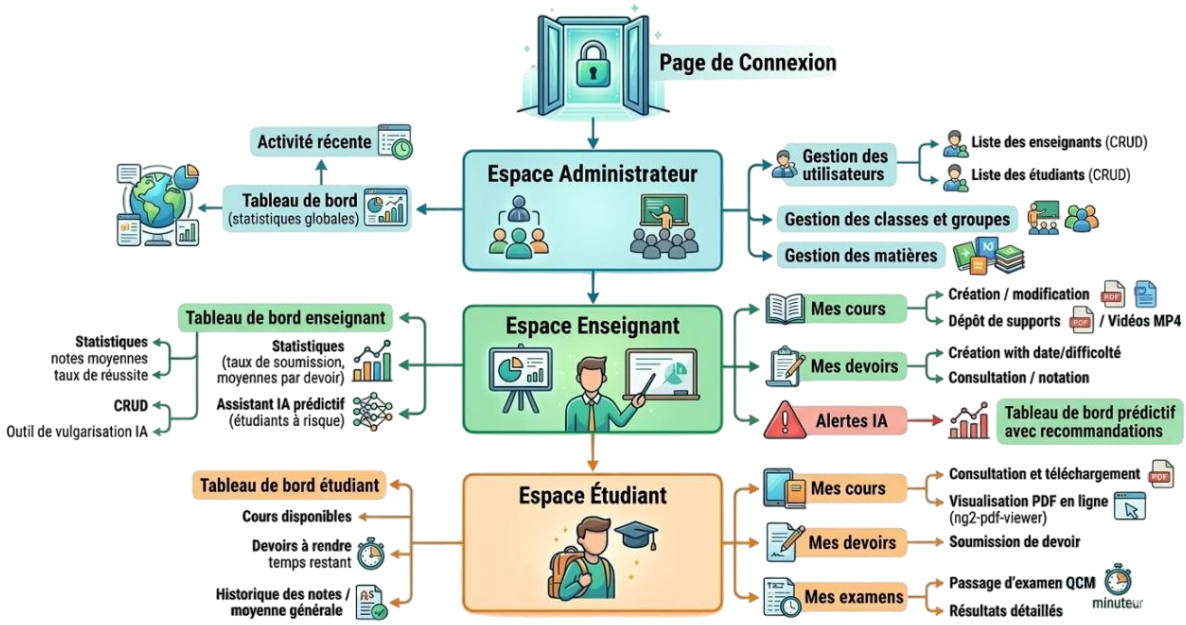
Les maquettes seront réalisées à l'aide d'un outil de mockup (Figma).

3. DESCRIPTION FONCTIONNELLE ET TECHNIQUE

3.1. Arborescence du site

```
Page de Connexion
│
├── Espace Administrateur
│   ├── Tableau de bord (statistiques globales, activité récente)
│   ├── Gestion des utilisateurs
│   │   ├── Liste des enseignants (CRUD)
│   │   └── Liste des étudiants (CRUD)
│   ├── Gestion des classes et groupes
│   └── Gestion des matières
│
├── Espace Enseignant
│   ├── Tableau de bord enseignant
│   │   ├── Statistiques (taux de soumission, moyennes par devoir)
│   │   └── Assistant IA prédictif (étudiants à risque)
│   ├── Mes cours
│   │   ├── Création / modification de cours
```

```
│   │   └── Dépôt de supports (PDF, vidéos MP4)
│   ├── Mes devoirs
│   │   ├── Création de devoirs (avec dates limites et difficulté)
│   │   └── Consultation / notation des soumissions
│   └── Alertes IA
│       └── Tableau de bord prédictif avec recommandations
│
└── Espace Étudiant
    ├── Tableau de bord étudiant
    │   ├── Cours disponibles
    │   ├── Devoirs à rendre (avec temps restant)
    │   └── Historique des notes / moyenne générale
    ├── Mes cours
    │   ├── Consultation et téléchargement des supports
    │   └── Visualisation PDF en ligne (ng2-pdf-viewer)
    └── Mes devoirs
        └── Soumission de devoir
```



Pages construites sur le même modèle :

- Toutes les pages de gestion CRUD (utilisateurs, classes, matières) suivent le même modèle : liste avec recherche + formulaire de création/édition.
- Les trois tableaux de bord suivent chacun un modèle spécifique adapté au rôle.

3.2. Description fonctionnelle

➤ Espace sécurisé par mot de passe

Oui. Chaque utilisateur (Admin, Enseignant, Étudiant) accède à un espace personnel sécurisé après authentification par email et mot de passe. Le backend génère un Token JWT contenant l'identifiant, le rôle et la date d'expiration. Ce token est stocké côté client (localStorage) et envoyé automatiquement avec chaque requête via un Interceptor Angular.

Autorisation RBAC : Côté Angular, des Guards empêchent la navigation vers des pages non autorisées selon le rôle. Côté Spring Boot, Spring Security vérifie la validité du token et le rôle à chaque requête reçue. Un token expiré, falsifié ou absent entraîne un rejet avec erreur 401.

Mots de passe : stockés en base sous forme de hash BCrypt (irréversible).

Règle fondamentale : la protection côté frontend est une commodité d'interface pour guider l'utilisateur. La vraie sécurité réside toujours côté backend, car le frontend s'exécute dans le navigateur de l'utilisateur et peut être contourné.

➤ **Back office de suivi d'activité**

Oui. Trois niveaux de back office sont prévus :

Administrateur :

- Gestion complète des comptes (création, modification, suppression)
- Configuration des classes, groupes et matières
- Consultation des statistiques globales (nombre total d'utilisateurs répartis par rôle, nombre de cours publiés, taux de réussite global, activité récente)

Enseignant :

- Gestion de ses propres cours et supports pédagogiques
- Gestion de ses devoirs avec suivi des soumissions
- Liste de ses cours avec le nombre de consultations
- Liste de ses devoirs avec le taux de soumission (soumissions reçues vs. nombre d'étudiants)
- Moyenne des notes par devoir
- Résultats de l'assistant IA prédictif (étudiants à risque avec recommandations)

Étudiant :

- Cours disponibles pour sa classe
- Devoirs à rendre avec temps restant
- Historique de ses notes et moyenne générale

Fonctionnalités détaillées par module

➤ **Module Gestion de Contenu (ECM) :**

- **Création de cours :** l'enseignant remplit un formulaire (titre, description, difficulté, matière, classe ciblée) et y attache des supports pédagogiques
- **Upload de fichiers :** les formats autorisés sont **PDF** (.pdf) et **vidéos** (.mp4). Le backend effectue une double validation :

    - Vérification du **type MIME réel** du fichier (pas seulement l'extension). Les fichiers exécutables (.exe, .zip, .bat) sont catégoriquement rejetés
    - Vérification de la **taille** : maximum 10 Mo pour un PDF, 50 Mo pour une vidéo
- **Stockage :** les fichiers physiques sont sauvegardés dans un dossier sur le serveur via l'interface MultipartFile de Spring Boot. Seul le chemin d'accès relatif est enregistré en base de données
- **Consultation :** l'étudiant accède aux cours de sa classe, peut télécharger les supports ou lire les PDF directement en ligne grâce au composant **ng2-pdf-viewer** intégré à Angular
- **UX :** lors de l'upload de fichiers lourds, un **Loader** (spinner) est affiché pour informer l'utilisateur que le transfert est en cours

➤ **Module Devoirs :**

- **Création par l'enseignant :** titre, description (consignes), difficulté, date de début, date de fin, fichier joint optionnel (énoncé), classe ciblée
- **Cycle de vie d'un devoir :**
    - **Avant la date de début :** le devoir est visible mais la soumission n'est pas encore ouverte
    - **Entre date de début et date de fin :** le devoir est actif, les étudiants peuvent soumettre leurs travaux
    - **Après la date de fin :** le devoir est fermé, aucune soumission n'est plus possible
- **Blocage strict des soumissions tardives** à deux niveaux :
    - **Côté Angular :** le bouton de soumission est visuellement désactivé (grisé) lorsque la date de fin est dépassée
    - **Côté Spring Boot :** le backend compare la date/heure actuelle du **serveur** avec la date de fin du devoir. Si dépassée, la soumission est rejetée
- La date de début doit obligatoirement être antérieure à la date de fin (vérification backend)
- Une seule soumission autorisée par étudiant et par devoir

- Notation manuelle par l'enseignant avec commentaires

➤ **Module Intelligence Artificielle — Assistant Pédagogique Prédictif :**

L'assistant pédagogique est le **cœur différenciateur** de cette plateforme. Son rôle est d'**alerter l'enseignant** lorsqu'un devoir à venir risque de provoquer un taux d'échec important, en identifiant nommément les étudiants à risque et en proposant des actions pédagogiques concrètes.

Cette fonctionnalité repose sur l'utilisation d'une **API LLM externe** à laquelle le backend envoie un prompt structuré contenant les données de performance.

Logique de fonctionnement :

1. **Collecte des données :** Le backend rassemble automatiquement depuis la base de données : la difficulté estimée du cours et du prochain devoir, la liste des étudiants avec moyenne, notes précédentes et taux de participation
2. **Construction du prompt :** Le backend construit une consigne stricte exigeant une réponse en JSON structuré
3. **Nettoyage de la réponse :** Le backend isole le JSON pur et le transforme en objets Java via ObjectMapper
4. **Restitution :** Angular affiche un tableau de bord coloré (risque élevé / modéré / faible) avec recommandations pédagogiques concrètes

Principe fondamental : *L'IA est un outil d'aide à la décision, pas un décideur.*

3.3. Informations relatives aux contenus

➤ **Types de contenus :**

- Supports de cours (PDF, vidéos MP4)
- Devoirs créés par les enseignants (avec fichiers joints optionnels)
- Fichiers soumis par les étudiants
- Notes et commentaires attribués par les enseignants
- Recommandations pédagogiques générées par l'IA
- Graphiques et indicateurs dans les tableaux de bord

➤ **Modification des contenus :**

Les enseignants doivent pouvoir créer, modifier et supprimer leurs cours, supports et devoirs de manière autonome. L'administrateur gère les utilisateurs et la configuration globale.

➤ **Standards SEO :**

Non prioritaire. L'application est entièrement sécurisée par authentification ; aucun contenu public n'est destiné à l'indexation.

3.4. Contraintes techniques

Technologies imposées

Couche	Technologie	Rôle
Frontend	Angular	Interface utilisateur web
Backend	Spring Boot	Serveur applicatif, logique métier, sécurité
Base de données	PostgreSQL	Stockage des données structurées

➤ **Compatibilité navigateurs**

Chrome, Firefox, Edge, Safari — dernières versions.

➤ **Intégrations à des systèmes tiers**

- API IA externe (OpenAI ou Google Gemini) pour l'analyse prédictive et la génération de recommandations pédagogiques.

Contraintes de sécurité

- Double validation : côté frontend ET côté backend
- Mots de passe hachés en BCrypt, tokens JWT avec expiration
- Configuration CORS restreinte à l'origine du frontend Angular
- Protection contre les injections SQL via JPA (requêtes paramétrées)
- DTO sécurisés excluant les réponses correctes pendant les examens
- Nettoyage et validation systématique des réponses JSON retournées par l'IA

4. PRESTATIONS ATTENDUES ET MODALITÉS DE SÉLECTION DES PRESTATAIRES

4.1. Prestations attendues

Le projet étant réalisé dans le cadre d'un stage, les prestations suivantes sont attendues du développeur :

- ☑ Développement (frontend Angular + backend Spring Boot + intégration IA)
- ☑ Design (interfaces utilisateur, tableaux de bord, maquettes)
- ☑ Intégration (connexion frontend-backend via API REST, intégration API IA)

➤ Livrables attendus :

- Code source versionné (Git) : projets Angular et Spring Boot
- Script SQL de création de la base de données avec données de test
- Rapport de stage complet
- Présentation de soutenance
- Application fonctionnelle déployée
- Documentation technique (README)

4.2. Planning

Le projet s'étale sur environ 12 semaines (1er Mars — 31 Mai), découpé en 5 phases.

Phase	Période	Objectifs clés
1. Cadrage & Conception	1er – 15 Mars	Analyse des besoins, conception BDD et diagrammes UML, maquettes des interfaces
2. Base & Fichiers	16 Mars – 15 Avril	Initialisation Angular + Spring Boot, entités JPA, authentification JWT, RBAC, CRUD cours, upload physique avec validation MIME, intégration ng2-pdf-viewer

Phase	Période	Objectifs clés
3. Devoirs & Logique métier	16 – 30 Avril	Module complet des devoirs : création, soumission, blocage des dates, désactivation du bouton Angular, notation manuelle
4. IA & Dashboards	1er – 15 Mai	Requêtes d'agrégation SQL, tableaux de bord par rôle, connexion API IA, construction du prompt, nettoyage JSON, affichage prédictif avec code couleur
5. Tests & Finitions	16 – 31 Mai	Tests de sécurité, robustesse IA, blocage des soumissions tardives, validation MIME, corrections UI/UX, finition du rapport, préparation de la soutenance

➤ **Date de livraison finale :** 31 Mai.

4.3. Méthodologie de suivi

- **Méthodologie agile :** Le développement suivra une approche itérative et incrémentale, avec des livraisons intermédiaires à la fin de chaque phase.
- **Suivi du projet :** Échanges réguliers avec l'encadrant de stage (réunions hebdomadaires, démonstrations des avancées).
- **Versionnement :** Code source versionné avec Git tout au long du projet.

Contact pour questions sur le cahier des charges : Le tuteur de stage et le stagiaire développeur.